Jeudi 22/10 — Recherche, tris et complexité

Kevin Bardoux

jeudi 22 octobre 2026

Table des matières

Objectifs 2
Matin — Recherche, tris & complexité (grand O) — tris à la main, puis en code 2
Rappel (5 min) 2
Apport 1 — Deux façons de chercher (15 min) 2
Chercher — attention et vérification 3
Exercice j1 — Recherche linéaire (15 min) 3
Apport 2 — Trier (10 min) 4
Trier — attention et vérification 4
Activité — Trier des cartes à la main (30 min) 4
Apport 3 — Combien d'étapes ? (15 min) 5
Grand O — attention et vérification 6
Si le temps le permet — Le tri par insertion, à la main 6
À retenir — ce matin 6
Débrief 7
Après-midi — TP — « Algorithmes de tri » 7
Consignes (2 h) 7
Les quatre fonctions 7
Faire le lien avec le grand O 7
Entraînement ECF sur papier (30 min) 8
À retenir — cet après-midi 9
Si le temps le permet — les bulles sur une liste triée 9
Pour aller plus loin 9
Sources 10

Note

Compétences visées : résolution de problème · communiquer (expliquer un algorithme) **Livrable** : `tri-dichotomie`, `tri-selection`, `tri-bulles` et `tri-comparaisons` au vert.

Objectifs

- Comparer la recherche linéaire et la recherche dichotomique.
- Trier à la main, par sélection et à bulles, en comptant les comparaisons.
- Dire ce que devient le nombre d'étapes quand la liste double.
- Coder une recherche dichotomique et deux tris.

Comment se passe la journée. Le matin, trois notions : chercher dans un tableau, trier, compter les étapes. Chacune suit le même ordre : pourquoi, définition, exemple, pas à pas, erreur fréquente, vérification ; un encadré « À retenir » termine chaque demi-journée. Les tris et le grand O ne sont **pas** au programme de l'ECF de demain.

Matin — Recherche, tris & complexité (grand O) — tris à la main, puis en code

Rappel (5 min)

1. Comment parcourir chaque valeur d'un tableau ?
2. Que renvoie `'Ada Lovelace'.split(' ')` ?
3. Que fait `${…}` dans un littéral de gabarit ?

Trois questions courtes, sans regarder vos notes : la veille, la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

Apport 1 — Deux façons de chercher (15 min)

- **Pourquoi** : retrouver une valeur, vite, dans une longue liste.
- **Linéaire** : case par case ; **dichotomique** : le milieu, une moitié.

Pourquoi. Chercher un client parmi un million demande une méthode efficace : le temps de recherche compte.

Définition. La **recherche linéaire** regarde les cases une par une, du début à la fin ; elle marche toujours, même si le tableau n'est pas trié. La **recherche dichotomique** regarde la case du milieu, puis ne garde que la moitié qui peut contenir la valeur, et recommence ; elle n'est possible que si le tableau est **trié** (Wikipédia, s. d.-a). Ces deux recherches sont celles de la *Lecture 3* du cours CS50 (CS50 2026), qui reprend l'annuaire de la première séance.

Exemple. Chercher `23` dans un tableau trié de 8 valeurs :

```
[3, 8, 12, 17, 23, 31, 40, 52]
linéaire     : 3, 8, 12, 17, 23        → 5 cases regardées
dichotomique : 17 (milieu), 31, 23     → 3 cases regardées
```

Pas à pas (dichotomique). Le formateur le fait au tableau ; vous le suivez sur votre feuille.

1. Bornes : de l'index `0` à l'index 7. Milieu : `Math.floor((0 + 7) / 2)` = 3, la valeur `17`.
2. `23` est plus grand que `17` : on garde la moitié droite, de l'index 4 à 7.
3. Milieu : `Math.floor((4 + 7) / 2)` = 5, la valeur `31`. `23` est plus petit : on garde de 4 à 4.
4. Milieu : 4, la valeur `23`. Trouvé, à l'index 4.

Chercher — attention et vérification

> ⚠ Attention
>
> La recherche dichotomique sur un tableau **non trié** donne un résultat faux, sans erreur affichée. Vérifiez d'abord que le tableau est trié.

- **Vérifiez** : combien de cases pour trouver `3`, en dichotomique ?

Exercice j1 — Recherche linéaire (15 min)

1. Ensemble (5 min) : la boucle `for` sur les index, de `0` à `length - 1`.

2. Seuls : **j1-recherche-lineaire**, l'index de la valeur, ou `-1`.
3. Sans `indexOf` ; les tests vérifient le tableau vide et l'absent.
4. **Réussi si** : tests au vert, aucune alerte ESLint.

Apport 2 — Trier (10 min)

- **Pourquoi** : la dichotomie exige un tableau trié.
- **Sélection** : la plus petite en premier ; **bulles** : échanger les voisines.

Pourquoi. Pour chercher vite, il faut des données triées ; pour afficher une liste (prix croissants, noms par ordre alphabétique), aussi.

Définition. Le **tri par sélection** cherche le plus petit élément restant et le place à sa position définitive, puis recommence avec le reste (Wikipédia, s. d.-d). Le **tri à bulles** compare les éléments voisins et les échange s'ils sont mal rangés ; il repasse tant qu'un passage a fait au moins un échange (Wikipédia, s. d.-b).

Exemple. Le tri par sélection sur `[5, 2, 4, 1]` :

```
[5, 2, 4, 1]   la plus petite est 1 : on l'échange avec 5
[1, 2, 4, 5]   dans le reste, la plus petite est 2 : déjà en place
[1, 2, 4, 5]   dans le reste, 4 : déjà en place. Trié.
```

Pas à pas. C'est l'activité des cartes, ci-dessous : le formateur montre un tri par sélection avec 4 cartes, puis vous le refaites par deux avec 8 cartes.

Trier — attention et vérification

> ⚠ Attention
>
> Pour échanger deux valeurs, il faut une variable temporaire : sinon, l'une écrase l'autre et une valeur est perdue.

- **Vérifiez** : triez `[3, 1, 2]` à bulles : combien d'échanges ?

Activité — Trier des cartes à la main (30 min)

1. Par deux, 8 cartes mélangées : un **trieur**, un **compteur**.

2. Tri par sélection : le compteur note chaque **comparaison**.
3. Tri à bulles, rôles échangés : même chose.
4. Notez les deux totaux : lequel compare le moins ?
5. **Réussi si** : vos deux totaux sont notés et comparés.

Le **trieur** déplace les cartes et dit chaque comparaison à voix haute (« 7 est plus grand que 3 ») ; le **compteur** fait une marque sur la feuille à chaque comparaison. Refaites ensuite le tri par sélection avec une carte de plus : combien de comparaisons en plus ?

Apport 3 — Combien d'étapes ? (15 min)

- **Pourquoi** : savoir si un programme reste rapide.
- **Grand O** : comment les étapes grandissent quand la liste grandit.

Pourquoi. Un programme rapide sur 10 valeurs peut devenir très lent sur un million. On compare les algorithmes en regardant ce qui se passe quand la quantité de données **double**.

Définition. Le « grand O » décrit l'ordre de grandeur du nombre d'étapes d'un algorithme, plutôt qu'un calcul exact (CS50 2026). Retenez l'intuition : ce qui compte, c'est ce qui se passe quand la liste grandit.

Exemple. Quand la liste double :

```
recherche linéaire      O(n)      deux fois plus
d'étapes
recherche dichotomique  O(log n)  une seule étape de
plus
tri par sélection       O(n²)     environ quatre fois
plus
```

Pas à pas. Avec les résultats de l'activité des cartes :

1. Tri par sélection, 4 cartes : 3 + 2 + 1 = 6 comparaisons.
2. 8 cartes (deux fois plus) : 28 comparaisons, presque cinq fois plus.
3. 16 cartes : 120 comparaisons, plus de quatre fois 28.

4. Conclusion : quand la liste double, les comparaisons font environ ×4 : c'est $O(n^2)$.

Grand O — attention et vérification

> ⚠ Attention
>
> Le grand O ne donne pas un temps en secondes : il dit comment le nombre d'étapes **grandit**. Un $O(n^2)$ devient vite très lent sur de grandes listes.

- **Vérifiez** : liste de 1 000 → 2 000 éléments, que devient $O(n^2)$?

Si le temps le permet — Le tri par insertion, à la main

1. Prenez les cartes en main, une par une, comme dans un jeu de cartes.
2. Glissez chaque nouvelle carte vers la gauche jusqu'à sa place.
3. Comptez les comparaisons sur des cartes déjà triées : que remarquez-vous ?

Le tri par insertion insère chaque élément à sa place parmi ceux déjà triés (Wikipédia, s. d.-c). Il est codé cet après-midi dans « Pour aller plus loin ».

À retenir — ce matin

> 💡 À retenir
>
> - **Chercher** : dichotomique seulement si le tableau est trié.
> - **Trier** : sélection ou bulles ; échanger avec une variable temporaire.
> - **Grand O** : ce que deviennent les étapes quand la liste double.

Le détail, notion par notion :

- **Chercher** : la recherche linéaire marche toujours ; la dichotomique regarde le milieu et garde une moitié, beaucoup plus vite, mais seulement sur un tableau trié.
- **Trier** : la sélection place la plus petite valeur à chaque tour ; les bulles échangent les voisines mal rangées ; un échange demande une variable temporaire.

- **Grand O** : O(n) double, O(log n) ajoute une étape, O(n^2) quadruple quand la liste double.

Débrief

- Pourquoi la dichotomie exige-t-elle un tableau trié ?
- Quel tri vous a semblé le plus simple à expliquer ?
- Que devient O(n^2) si la liste passe de 1 000 à 2 000 ?

Après-midi — TP — « Algorithmes de tri »

Consignes (2 h)

1. Quatre fonctions, dans l'ordre, fichiers `tri-….`
2. Pour chacune : refaites l'algorithme avec vos cartes, puis codez.
3. Travaillez sur une copie : `array.slice()`.
4. Interdits : `sort`, `indexOf`, `includes` sur le tableau à trier.
5. **Réussi si** : les quatre au vert, aucune alerte ESLint.

Les quatre fonctions

- **tri-dichotomie** (moyen) : l'index de la valeur dans un tableau trié.
- **tri-selection** (moyen) : un nouveau tableau trié.
- **tri-bulles** (moyen) : un nouveau tableau trié, autrement.
- **tri-comparaisons** (moyen) : compter les comparaisons.

`slice()` sans argument renvoie une copie du tableau (MDN Web Docs, s. d.). Pour échanger deux cases, utilisez une variable temporaire : `const temporary = result[i]; result[i] = result[j]; result[j] = temporary;`.

> Indice — tri-dichotomie
>
> Gardez deux bornes, `low = 0` et `high = sortedArray.length - 1`. Tant que `low <= high` : calculez le milieu avec `Math.floor((low + high) / 2)`, comparez, puis déplacez `low` ou `high` juste après ou juste avant le milieu.

Faire le lien avec le grand O

1. `tri-comparaisons` au vert : ajoutez les 3 lignes de la page.
2. Lancez `node tri-comparaisons.js` : 6, 28, puis 120.

3. Liste doublée : environ ×4. C'est O(n^2).
4. Effacez les 3 lignes, puis relancez les tests.

Les trois lignes, à coller tout en bas de `tri-comparaisons.js`, après l'accolade qui ferme la fonction :

```
console.log(selectionSortComparisons([4, 3, 2, 1]));
console.log(selectionSortComparisons([8, 7, 6, 5, 4, 3,
2, 1]));
console.log(selectionSortComparisons([16, 15, 14, 13,
12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]));
```

Vous devez voir :

```
PS C:\Users\stagiaire\dwwm\semaine-03> node tri-
comparaisons.js
6
28
120
```

Si vous voyez `À faire (4,3,2,1)`, la fonction n'est pas encore écrite : faites d'abord passer ses tests.

Entraînement ECF sur papier (30 min)

1. Sans ordinateur, seul, comme demain matin.
2. Trois exercices : prédire, trouver un bug, écrire.
3. Puis correction collective (15 min).

1. Qu'affiche ce programme ? Écrivez la valeur de `result` après chaque tour.

```
const words = ['sun', 'moon', 'star'];
let result = '';
for (let i = words.length - 1; i >= 0; i--) {
  result = result + words[i][0];
}
console.log(result);
```

2. Trouvez le bug. `average([4, 6, 8])` devrait renvoyer 6. Que renvoie-t-elle ? Corrigez-la. Que renvoie `average([])` ? Faites-lui renvoyer `null` dans ce cas.

```
function average(numbers) {
  let sum = 0;
  for (let i = 1; i < numbers.length; i++) {
    sum = sum + numbers[i];
  }
  return sum / numbers.length;
}
```

3. Écrivez à la main la fonction `countLongWords(words, minLength)`, qui renvoie le nombre de mots d'au moins `minLength` lettres : `countLongWords(['a', 'abc', 'abcd'], 3)` renvoie 2.

À retenir — cet après-midi

> À retenir
>
> - **Copie** : `array.slice()` ; le tableau reçu reste intact.
> - **Dichotomie** : `low <= high`, puis `middle + 1` ou `middle - 1`.
> - **Échange** : `const temporary = a[i];` d'abord.

Critères de réussite

- □ Les tests des quatre fonctions `tri-…` sont au vert
- □ Aucune fonction n'utilise `sort` ni ne modifie le tableau reçu
- □ Vous savez expliquer pourquoi le tri par sélection est en $O(n^2)$
- □ L'entraînement papier est fait et corrigé

Si le temps le permet — les bulles sur une liste triée

1. Comptez les comparaisons du tri à bulles sur `[1, 2, 3, 4]`.
2. Puis sur `[4, 3, 2, 1]`. Expliquez la différence.

Pour aller plus loin

- **tri-insertion** : le tri par insertion, avec ses tests ; même règle, un nouveau tableau, sans `sort`.
- Fiche facultative : « Les algorithmes », en deux parties de 15 min.
- VisuAlgo : les tris de la journée, animés pas à pas.

La fiche de lecture « Les algorithmes », d'après la leçon 3 de CS50 :

1. chercher, mesurer

(recherche, grand O, Ω et Θ) et

2. trier, récursivité

(tris, récursivité, tri par fusion). Ses deux exercices sont au niveau 3 du parcours de remédiation de mercredi.

VisuAlgo, de l'université nationale de Singapour, anime les tris à bulles, par sélection et par insertion sur une liste que vous choisissez, comparaison après comparaison (Steven Halim (National University of Singapore), s. d.). Gratuit, sans compte, en anglais. Comptez les comparaisons affichées et comparez-les à celles de `tri-comparaisons`.

Sources

CS50. 2026. « Lecture 3 - CS50x 2026 ». https://cs50.harvard.edu/x/notes/3/.

MDN Web Docs. s. d. « Array : méthode slice() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/slice.

Steven Halim (National University of Singapore). s. d. « Sorting (Bubble, Selection, Insertion, Merge, Quick, Counting, Radix) — VisuAlgo ». Consulté le 3 octobre 2026. https://visualgo.net/en/sorting.

Wikipédia. s. d.-a. « Recherche dichotomique ». Consulté le 26 septembre 2026. https://fr.wikipedia.org/wiki/Recherche_dichotomique.

Wikipédia. s. d.-b. « Tri à bulles ». Consulté le 26 septembre 2026. https://fr.wikipedia.org/wiki/Tri_%C3%A0_bulles.

Wikipédia. s. d.-c. « Tri par insertion ». Consulté le 26 septembre 2026. https://fr.wikipedia.org/wiki/Tri_par_insertion.

Wikipédia. s. d.-d. « Tri par sélection ». Consulté le 26 septembre 2026. https://fr.wikipedia.org/wiki/Tri_par_s%C3%A9lection.